Piloto FES para Rootstock

Informe técnico para agente de ingeniería: lógica, datos críticos, diagnóstico corregido del monitor de staking y plan de resolución

Andromeda Core

Octubre de 2026

Misión del agente y cambio de diagnóstico (leer primero)

Misión. Investigar a fondo el repositorio del piloto (`andromeda-rootstock` / `coreV`, dominio `core.andromedacomputer.net`), verificar cada hallazgo de este documento contra el código real y la cadena, y dejar el piloto en condiciones de abrir la ventana de observación de 4 semanas.
Qué cambió. La auditoría del 30 de septiembre atribuía el fallo del monitor de staking a configuración (cron ausente y variables de entorno). El análisis del 20 de octubre muestra que **la causa raíz es otra**: el monitor está construido sobre un evento `Staked(address,uint256,uint256)` que **no existe** en el contrato de Rootstock Collective. Arreglar el cron y las variables **no produce ningún resultado**: hay que **reescribir la detección** usando eventos `Transfer` ERC20 (sección 5).
Regla de oro. Todo dato de este documento es una afirmación a verificar, no un hecho probado por el agente. La sección 11 lista lo que **no** está confirmado.

Regla de verificación antes de publicar (obligatoria)

Todo dato numérico o criptográfico que vaya al dashboard, al foro o al reporte final debe verificarse contra una **fuente externa** antes de publicarse: el snapshot en IPFS, el contrato en el explorador de Rootstock, el ABI oficial. **No** contra este documento, no contra la memoria de la conversación y no contra lo que "suena correcto". Este informe es un mapa de lo que hay que comprobar, no una fuente de verdad. La tabla de la sección 10 indica la fuente externa de cada dato.

Contenido

1 Hoja de datos críticos **2**

2 Lógica del piloto **2**

2.1 Qué es y qué prueba . . . 2
2.2 Objetivos . . . 3
2.3 Flujo previsto y puntos de ruptura . . . 3

3 Hallazgos y evidencia **3**

4 Asignación de cohortes: diseño vs. realidad **5**

5 Monitor de staking: reescritura de la detección **5**

5.1 El supuesto equivocado . . . 5
5.2 Nueva lógica de detección . . . 6
5.3 Esquema de consulta (referencia, solo ASCII) . . . 6
5.4 Riesgos de la nueva detección . . . 6

6 Reconciliación del repositorio **7**

7 Plan de ejecución para el agente 7

8 Lo que debe y no debe hacer el sistema 8

9 Limitaciones del experimento 8

10 Verificación contra fuentes externas 8

11 Supuestos no verificados (el agente debe confirmarlos) 9

1 Hoja de datos críticos

Datos que no deben equivocarse

Proxy stRIF (usar para leer)	`0x5Db91E24BD32059584bbdB831a901F1199f3D459` Blockscout: "Staked RIF", proxy ERC1967, verificado. El proxy **no cambia** con los upgrades.
Implementación vigente	`0xC92B149fCDcf176D7e7A41eb51f56668B7BABe03` (StRIFTokenV02)
Implementación obsoleta	`0xcC13a032...` ("StRIFToken impl" de una lista previa). **No usar.**
Eventos del ABI V02	10 en total: `Approval`, `OwnershipTransferred`, `CollectiveRewardsAddress...`, `STRIFCollectiveRewardsErrorSkipChangedTo`, `DelegateChanged`, `Transfer`, `DelegateVotesChanged`, `Upgraded`, `EIP712DomainChanged`, `Initialized`. **Ninguno es Staked.**
topic0 de Transfer	`0xddf252ad1be2c89b69c2b068fc378daa952ba7f163c4a11628f55a4df523b3ef` = keccak256(`Transfer(address,address,uint256)`)
Señal de stake / unstake	stake: `to == proxy`; unstake: `from == proxy` (según el análisis; verificar en la sección 5)
Cohorte calificada real	**16 wallets** (no 19, cifra publicada por error en el foro): Grupo A = 8, Grupo B = 7, VIP = 1
Wallet VIP	9.24M RIF, 530 días inactiva. Umbral VIP: balance > 1,000,000 RIF
Outcome gate	50,000 RIF incremental atribuido (= 0.54 % del balance de la wallet VIP)
Ventana	4 semanas desde la publicación del enlace; **aún no iniciada**
Presupuesto	$11,200 USD (Stages 1–4); tramo de 32 % ($3,600) ligado al gate
Ramas git	`origin/main` = `fc5d2b09` (6 commits de Nicolás, **producción**); `origin/solana-hackathon` = `9e9b29d` (14 commits de Ilich). **Sin ancestro común.**
Tráfico actual	0 visitas orgánicas a `/preview`

2 Lógica del piloto

2.1 Qué es y qué prueba

FES (*Framework for Engagement & Staking*) es un experimento controlado de 4 semanas. Pregunta: **un reporte personalizado** (balance, yield no ganado, builders recomendados) **convierte mejor a holders inactivos de RIF en stakers que un mensaje genérico?** El holder ingresa su wallet en `https://core.andromedacomputer.net/en/preview` y ve el mensaje de su cohorte.

Cohorte	Wallets	Tratamiento
A (control)	8	Mensaje genérico
B (tratamiento)	7	Reporte personalizado (balance, yield, builders)
VIP	1	Balance > 1,000,000 RIF

2.2 Objetivos

- Medir si el reporte personalizado supera al mensaje genérico en conversión a staking.
- Alcanzar el outcome gate de 50,000 RIF incremental atribuido dentro de la ventana.
- Publicar un reporte final auditable en el foro de Rootstock, sin costo para la DAO hasta que haya resultados (autofinanciado por Andromeda Core).

2.3 Flujo previsto y puntos de ruptura

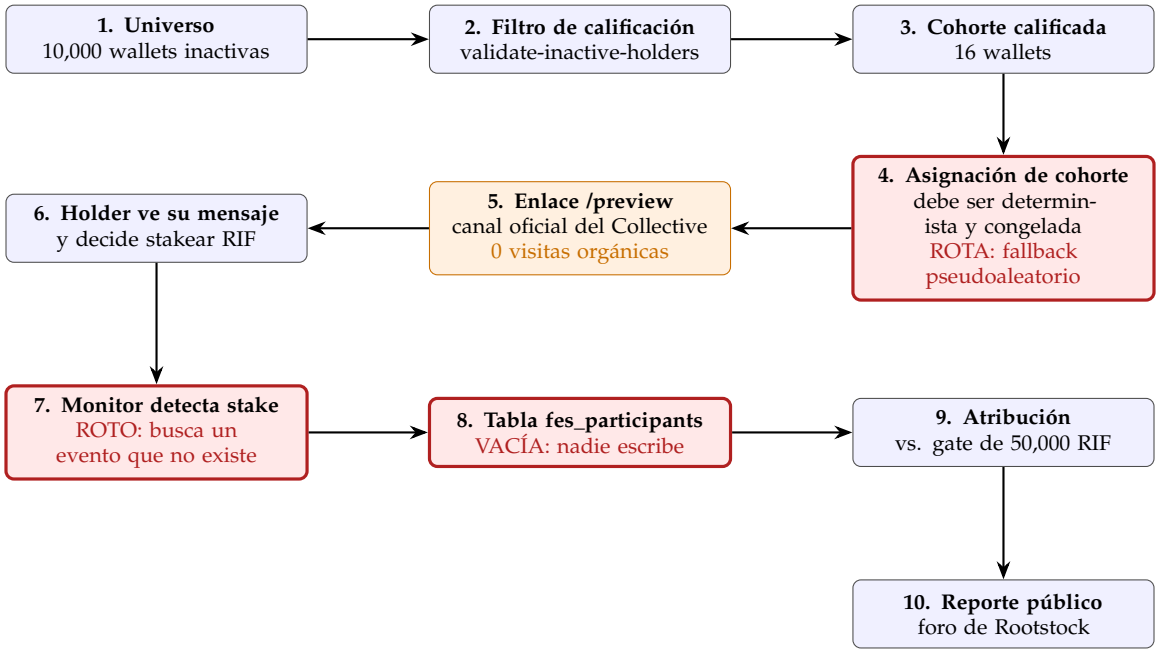


Figura 1: Pipeline del experimento. En rojo, los pasos que hoy no cumplen su función; en naranja, el cuello de botella de distribución.

3 Hallazgos y evidencia

Severidades: **P0** bloquea el lanzamiento o expone datos; **P1** invalida el experimento; **P2** deuda o riesgo menor.

ID	Sev.	Hallazgo	Qué verificar / acción esperada
M1	P0	El monitor asume el evento `Staked(address,uint256,uint256)`, inexistente en stRIF. **Causa raíz** de que el monitor nunca detecte nada.	Localizar el código del monitor y su cron handler; reescribir la detección con `Transfer` (sección 5).
M2	P0	`STAKED_EVENT_SIG` es un hex inventado.	Eliminar; usar el topic0 de Transfer de la hoja de datos.
M3	P0	`STAKING_CONTRACT_ADDRESS` es un placeholder.	Configurar con el proxy `0x5Db91E24...3D459` (nunca con la implementación).
M4	P0	No hay cron en `vercel.json`.	Añadir el cron solo después de rehacer la detección.
M5	P1	`eth_getLogs` falla en RPCs públicos probados: `public-node.rsk.co`, blastapi (403), tenderly (404), sks (SSL), drpc (400).	Elegir un RPC con logs (Alchemy, Infura u otro con soporte para Rootstock) o la API de Blockscout (responde, más lenta).
C1	P0	Fallback `assignSingleHolder` activo en producción (Redis caído): cohorte = `daysInactive` mod 2. Nunca produce VIP y no es el diseño.	Eliminar el fallback; asignación determinista desde una lista congelada (script de freeze).
C2	P0	El código emite la cohorte `WHALE` en vez de `VIP`; viola el CHECK constraint de Supabase.	Unificar etiquetas de cohorte en todo el código y en el schema.

ID	Sev.	Hallazgo	Qué verificar / acción esperada
C3	P0	Nadie llama a `batchUpsertParticipants` / `upsertParticipant`; `fes_participants` está vacía.	Trazar los call sites; poblar con las 16 wallets antes de publicar.
E1	P0	`NEXT_PUBLIC_SUPABASE_URL` se inlinea en build-time.	Redeploy tras fijar la variable; verificar el valor dentro del bundle.
E2	P0	El cliente usa la anon key en vez de `SUPABASE_SERVICE_ROLE_KEY`.	Escrituras de servidor con service role (solo servidor, nunca expuesta al cliente); revisar RLS.
E3	P1	Redis caído o eliminado.	Restaurar o eliminar la dependencia; nada crítico debe depender de un fallback silencioso.
S1	P0	`wallets.csv` (10,000 wallets) en el repo público.	Retirar del árbol y del historial; evaluar impacto.
S2	P0	API key de The Graph hardcodeada en `scripts/validate-inactive-holders.mjs`.	Rotar la clave, moverla a variable de entorno, limpiar el historial.
S3	P0	`/api/fes/participants` sin autenticación.	Proteger con el auth de `fes_admin.ts`.
S4	P0	El dashboard interno perdió su login: un force-push a `main` el 6 de agosto borró ese código.	Reestablecer auth (ver G1); prohibir force-push a `main`.
G1	P1	Ramas `main` y `solana-hackathon` sin ancestro común; producción corre `main`.	Reconciliación explícita (sección 6). Decisión tomada: adoptar `fes_admin.ts` (Nicolás) sobre `fes_dashboard_session` (Ilich).
D1	P1	Cifra publicada de 19 wallets; la real es 16.	Corregir en el foro antes de abrir la ventana.
D2	P1	0 visitas orgánicas a `/preview`: sin distribución no hay nada que atribuir, aunque el monitor quedara perfecto.	No es de código; es el cuello de botella real del lanzamiento.

Dependencias entre hallazgos

M3, M4 y M2 son irrelevantes mientras M1 no se corrija: con la dirección y la firma correctas de un evento `Staked` igualmente se encontrarían **cero** eventos, para siempre. C1, C2 y C3 forman una cadena: aunque el fallback se arreglara, la etiqueta `WHALE` haría fallar el insert, y aun sin eso nadie llama al insert.

4 Asignación de cohortes: diseño vs. realidad

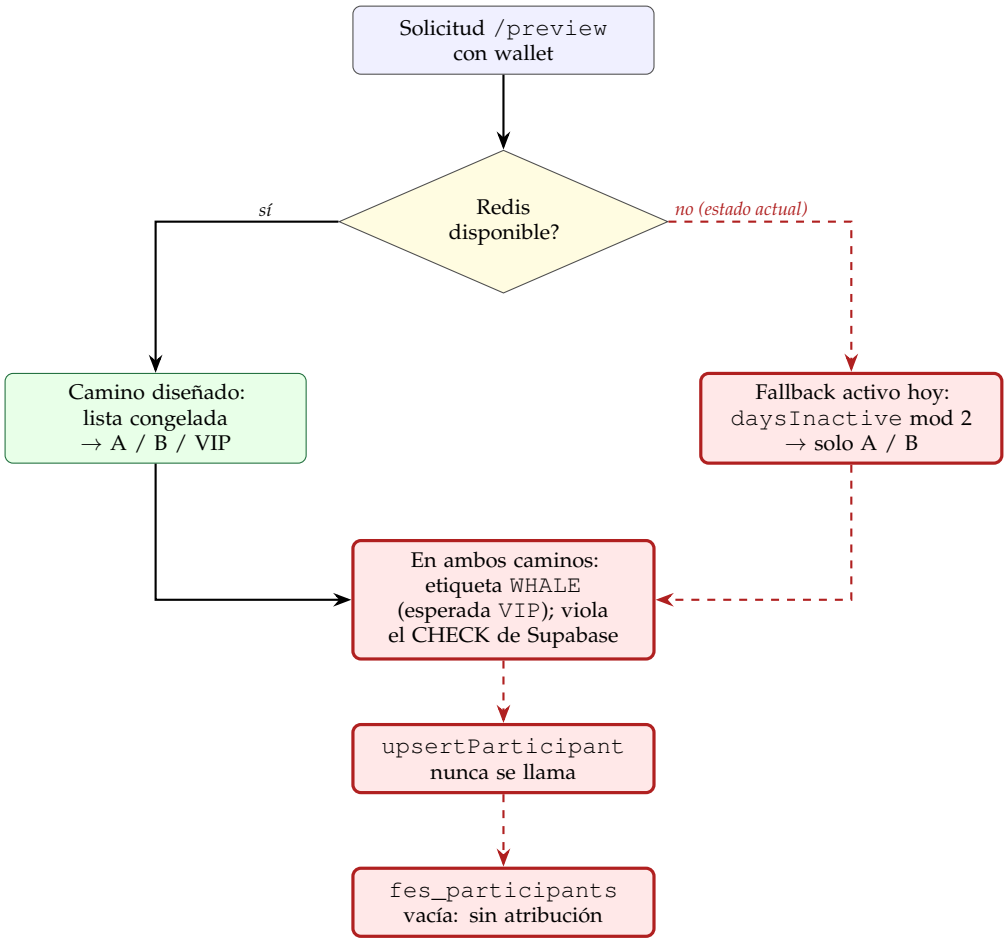


Figura 2: Aun con Redis funcionando, la etiqueta incorrecta y la ausencia de llamadas de escritura dejan la tabla vacía.

Criterio de aceptación. Cada una de las 16 wallets tiene una cohorte fija (8/7/1), idempotente ante recargas, persistida en `fes_participants` y consultable sin Redis.

5 Monitor de staking: reescritura de la detección

5.1 El supuesto equivocado

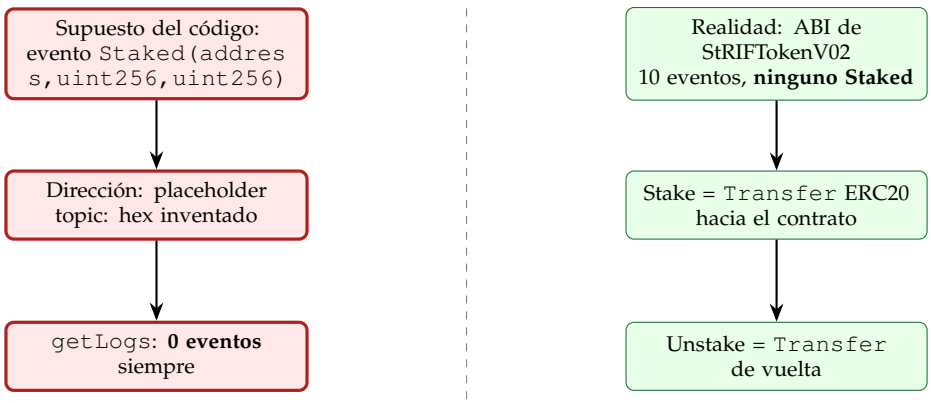


Figura 3: Izquierda: el monitor actual. Derecha: cómo se manifiesta el staking en la cadena.

5.2 Nueva lógica de detección

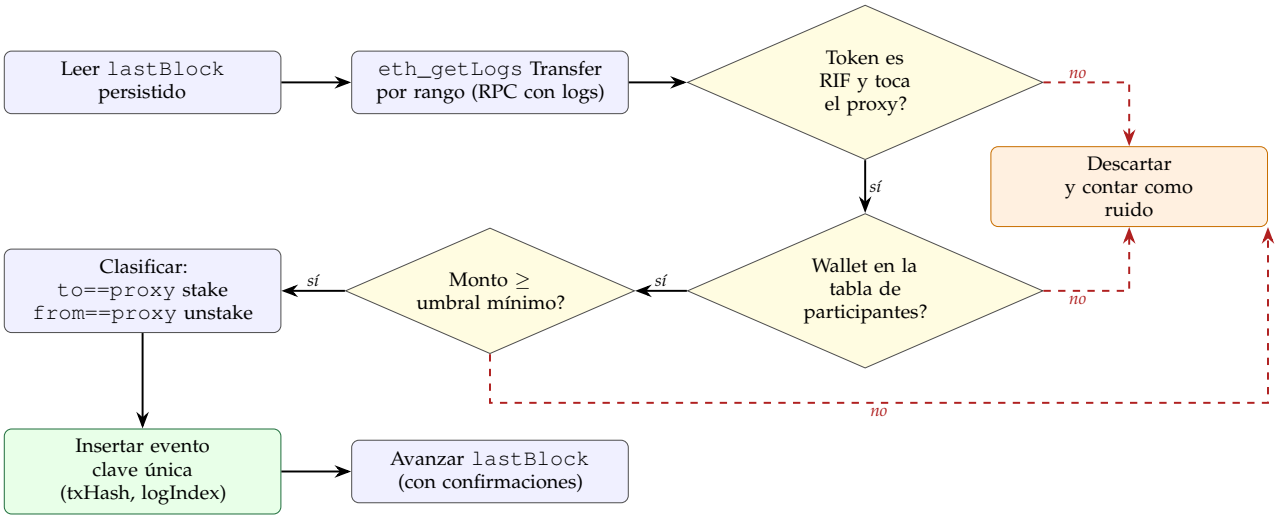


Figura 4: Detección propuesta. Cada rama de descarte debe registrarse para poder auditar falsos positivos.

5.3 Esquema de consulta (referencia, solo ASCII)

```
// Valores de la hoja de datos
const T0    = "0xddf252ad1be2c89b69c2b068fc378daa952ba7f163c4a11628f55a4df523b3ef";
const STRIF = "0x5Db91E24BD32059584bbdB831a901F1199f3D459"; // proxy
const RIF   = "<direccion del token RIF en Rootstock: VERIFICAR en Blockscout>";
const pad   = a => "0x" + a.slice(2).toLowerCase().padStart(64, "0");

// Stake: RIF transferido hacia el proxy
getLogs({ address: RIF, topics: [T0, null, pad(STRIF)], fromBlock, toBlock });
// Unstake: RIF transferido desde el proxy
getLogs({ address: RIF, topics: [T0, pad(STRIF), null], fromBlock, toBlock });
// Contraprueba: mint/burn de stRIF en el propio proxy
getLogs({ address: STRIF, topics: [T0, pad(ZERO), null], fromBlock, toBlock }); // mint
getLogs({ address: STRIF, topics: [T0, null, pad(ZERO)], fromBlock, toBlock }); // burn
```

5.4 Riesgos de la nueva detección

- **Falsos positivos.** Transferir RIF al contrato no siempre es stakear. Comprobar que el token es el RIF correcto, y evaluar un umbral mínimo.
- **Unstake vs. salida normal.** Un `Transfer` desde el contrato puede no ser unstake. Distinguirlos contrastando con mint/burn de stRIF y, si hace falta, con la función invocada en la transacción.
- **Atribución temporal.** Un stake solo es atribuible si ocurre *después* de que la wallet vio su mensaje. Esto exige registrar la vista; hoy no consta que se haga (sección 11).
- **RPC.** Ver M5. Fijar tamaño de rango, reintentos y manejo de reorganizaciones.
- **Idempotencia.** El cron de Vercel puede ejecutarse dos veces; la clave única (txHash, logIndex) evita duplicados.

Criterio de aceptación. Con una transacción de stake real conocida (usar `local-snapshots/verify-staking.mjs`, ignorado por git), el monitor detecta el evento, lo asocia a la wallet y a su cohorte, y lo registra una sola vez.

6 Reconciliación del repositorio

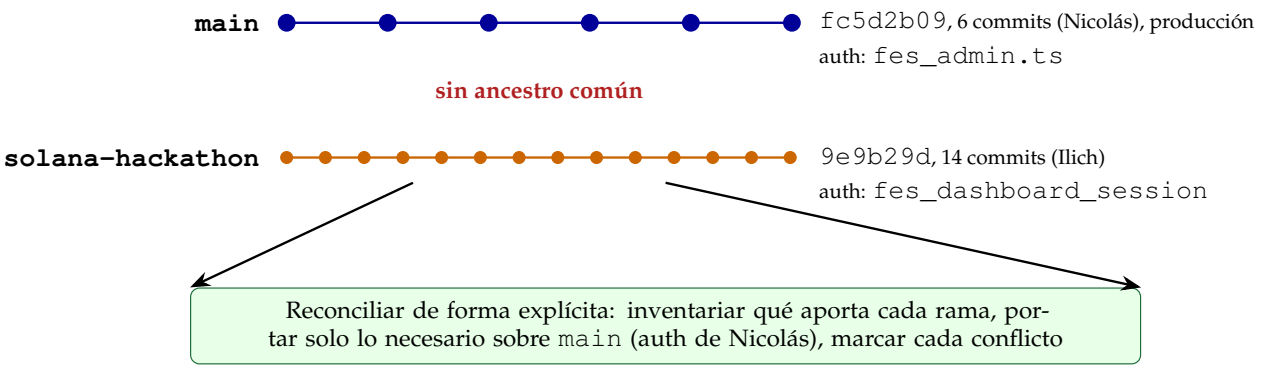


Figura 5: Las dos ramas no comparten historia; producción corre `main`.

Cuando hay varios agentes con acceso de escritura al mismo repositorio (hay otro agente trabajando en paralelo desde otra máquina), toda reconciliación debe ser explícita: `git fetch`, comparar el estado remoto antes de tocar nada, listar los conflictos y no hacer *force-push* jamás sobre `main`.

7 Plan de ejecución para el agente

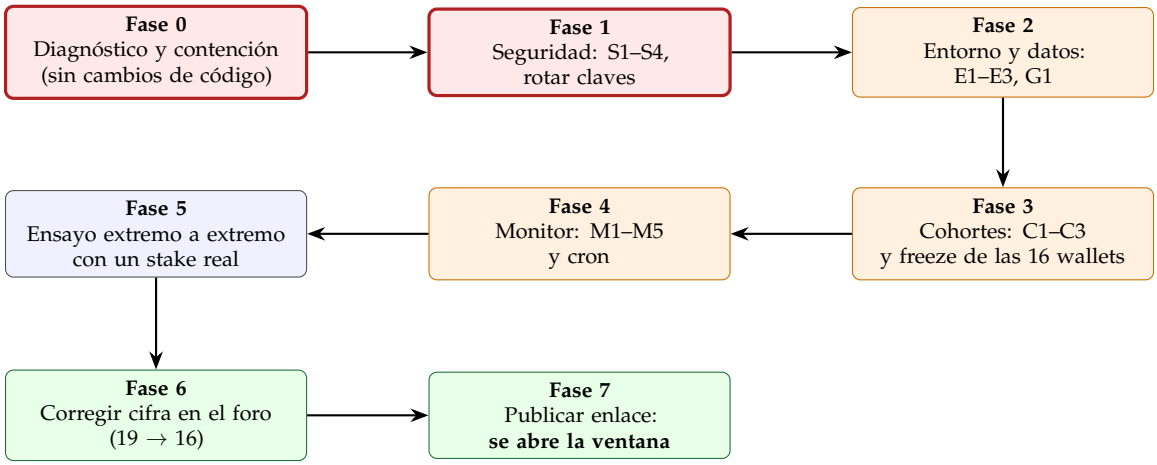


Figura 6: Orden de resolución. No publicar el enlace antes de la fase 7: el reloj de 4 semanas corre desde la publicación.

F0 Inventario sin modificar: ramas, estado de despliegue en Vercel, variables presentes (sin leer sus valores), schema y datos de Supabase, historial del force-push del 6 de agosto.

F1 Retirar `wallets.csv` y la API key del código, rotar la clave de The Graph, autenticar `/api/fes/participants`.

F2 Fijar `NEXT_PUBLIC_SUPABASE_URL` antes del build; separar service role (servidor) de anon key (cliente); decidir Redis; reconciliar ramas.

F3 Unificar cohortes, quitar el fallback, congelar y cargar las 16 wallets.

F4 Elegir RPC, implementar la detección por `Transfer`, añadir el cron.

F5 Ensayo con una transacción real; revisar el registro de descartes.

F6–F7 Corregir la cifra, avisar a Rootstock, publicar el enlace.

Reglas de trabajo

- **Diagnóstico antes que corrección**: comandos de lectura primero, cambios después.
- Para reemplazos exactos de texto usar scripts `python3` con heredoc, no `sed`.
- Antes de cada push: `yarn build 2>&1 | tail -N` debe terminar sin errores.
- **Nunca** pedir ni imprimir valores de secretos; leer las variables con `source .env.local` y referenciarlas por nombre. Los secretos ya expuestos se consideran comprometidos y se rotan.
- Sin *force-push* a `main`. Tras cada cambio, informar qué se tocó y qué queda abierto.

8 Lo que debe y no debe hacer el sistema

Debe	No debe
Asignar cohortes de forma determinista y congelarlas antes de publicar.	Usar el fallback `daysInactive` mod 2 ni la etiqueta `WHALE`.
Registrar cada participante en `fes_participants`.	Abrir la ventana de 4 semanas antes de cerrar los hallazgos P0.
Detectar stake y unstake mediante `Transfer` sobre el proxy correcto.	Buscar un evento `Staked` ni usar la implementación como dirección.
Contar solo wallets del piloto, tras haber visto su mensaje.	Contar como atribuido todo RIF que llegue al contrato.
Mantener autenticación en el dashboard y en todos los endpoints con datos.	Exponer `wallets.csv`, claves de API ni endpoints abiertos.
Publicar el reporte final en el foro dentro de la ventana.	Costar nada a la DAO antes de los resultados.

9 Limitaciones del experimento

Advertencias metodológicas

- **Poder estadístico nulo.** Con 8 wallets en control y 7 en tratamiento, una sola wallet que stakee cambia la conversión 12 a 14 puntos porcentuales. El resultado no puede presentarse como prueba causal; es un piloto exploratorio.
- **El gate depende de una sola wallet.** Con 9.24M RIF, la wallet VIP supera el gate de 50,000 RIF con solo el 0.54 % de su balance. Definir con Rootstock si el VIP cuenta para el gate o se reporta aparte.
- **Condiciones económicas.** Lo registrado dice “sin costo para la DAO hasta que haya resultados” y a la vez fija un presupuesto de $11,200 con $3,600 ligados al gate. Confirmar por escrito qué se paga, cuándo y bajo qué condición.
- **Distribución.** Con 0 visitas orgánicas no hay tratamiento aplicado; el cuello de botella actual no es de código.

10 Verificación contra fuentes externas

Ningún valor de la columna izquierda se publica hasta quedar contrastado con la fuente de la columna derecha, y la verificación debe quedar registrada (fecha, fuente, resultado).

Dato a publicar	Fuente externa de verificación
Dirección del proxy stRIF e implementación vigente	Explorador de Rootstock (Blockscout): pestaña del contrato, slot de implementación del proxy ERC1967 y código verificado.

Dato a publicar	Fuente externa de verificación
Lista de eventos y firmas (topic0)	ABI oficial verificado del contrato en Blockscout; recalcular keccak256 de cada firma de forma independiente.
Dirección del token RIF	Explorador de Rootstock y documentación oficial de Rootstock; no tomarla de este documento.
Cohorte calificada (16 wallets; 8/7/1)	Snapshot en IPFS (verificar el CID) recalculado contra balances en la cadena; no contra `wallets.csv` ni contra cifras previas del foro.
Balance y días de inactividad (p. ej. VIP 9.24M RIF, 530 días)	Consulta directa a la cadena (balances y historial de transferencias en el explorador).
Stakes y unstakes atribuidos	Transacciones y logs en el explorador (hash de cada transacción). Cada stake publicado debe poder abrirse en Blockscout.
Total incremental frente al gate de 50,000 RIF	Suma recalculada desde los logs en cadena, no desde `fes_participants`.

11 Supuestos no verificados (el agente debe confirmarlos)

1. Que el ABI de StRIFTokenV02 en Blockscout tenga exactamente los 10 eventos listados y ninguno sea `Staked`; verificar de forma independiente.
2. Qué contrato emite el `Transfer` relevante: el token RIF (`to == proxy`) o el propio proxy stRIF (mint/burn). Contrastar con una transacción de stake real antes de fijar el filtro.
3. La dirección exacta del token RIF en Rootstock mainnet, que este documento no fija.
4. Si hay umbral mínimo de stake y cuál, según la mecánica del Collective.
5. Que la distribución de las 16 wallets (8/7/1) coincida con el criterio de calificación en el código; recalcularla.
6. Si las vistas de `/preview` quedan registradas con marca de tiempo (necesario para atribuir).
7. Qué parte del código de `solana-hackathon` no existe en `main` y debe portarse.
8. El estado actual de cada hallazgo: la información base data del 30 de septiembre y puede haber cambiado.